

Day3_AM

超サポ
愉快カンパニー

アシスト

QAシート

- 気になったことについては気軽にQAシートに記入してください
 - https://docs.google.com/spreadsheets/d/1MxqWSRFn6r3BCzAa17TtrWa_HoRhxp0aUoN6Su5KDb0/edit?usp=drive_link

作成したEC2インスタンスを確認

i-05b748c60ff44d0be (p04-yamamoto10152-Linux-ssh) のインスタンス概要 情報

接続 インスタンスの状態 ▼ アクション ▼

less than a minute 前に更新済み

インスタンス ID i-05b748c60ff44d0be	パブリック IPv4 アドレス [REDACTED]	プライベート IPv4 アドレス [REDACTED]
IPv6 アドレス [REDACTED]	インスタンスの状態 ① 保留中	DNS [REDACTED]
ホスト名のタイプ IP 名: ip-10-0-5-29.ap-northeast-1.compute.internal	プライベート IP DNS 名 (IPv4 のみ) ip-10-0-5-29.ap-northeast-1.compute.internal	
プライベートリソースの DNS 名に応答 -	インスタンスタイプ t2.medium	Elastic IP アドレス [REDACTED]
自動的に割り当てられた IP アドレス -	VPC ID vpc-02243d68ba98a0677 (p04-yamamoto10152-vpc-01) ↗	AWS Compute Optimizer の検出結果 このインスタンスで利用可能な推薦はありません。
IAM ロール -	サブネット ID subnet-077652917c2ed2e46 (p04-subnet-public1) ↗	Auto Scaling グループ名 [REDACTED]
IMDSv2 Required	インスタンス ARN arn:aws:ec2:ap-northeast-1:958533308904:instance/i-05b748c60ff44d0be	管理対象 False
オペレーター -		

パブリックIPアドレスを確認してメモしておく

このIPアドレスは毎朝変わるため
このフェーズの間は毎朝確認して
設定ファイルを変更する

辞書型

辞書とは？～名前でデータを管理する箱～

Key(名前)とValue(値)のペアでデータを管理します

辞書とは？

Key(名前)とValue(値)の
ペアでデータを管理する箱

例：社員情報

```
{"name": "山田",  
  "age": 25,  
  "dept": "営業部"}
```

日常例で考えると

 英和辞典のイメージ

apple → りんご

banana → バナナ

orange → みかん

単語 (Key) を引くと
意味 (Value) が出てくる！



Keyという名前でValueを取り出せる！
番号ではなく意味のある名前でアクセスできるのが辞書の特徴

リストとの違い～どういうときに使うか～

番号より名前でアクセスしたいとき → 辞書の出番！

リスト（番号でアクセス）

```
person = ["山田", 25, "営業部"]
```

何番目が覚えておく必要がある

```
person[0] → 山田
```

```
person[1] → 25
```

```
person[2] → 営業部
```

😓 0番が名前か年齢かわかりにくい...

辞書（名前でアクセス）

```
person = {  
    "name": "山田",  
    "age": 25,  
    "dept": "営業部"  
}
```

```
person["name"] → 山田
```

```
person["age"] → 25
```

😊 名前で取り出せてわかりやすい！

辞書の作り方 (Key・Value)

{ } を使ってKey: Valueのペアを定義します

```
# 辞書の作り方: { } で囲み、Key: Value の形で書く
person = {
    "name": "山田",      # Key: "name"  Value: "山田"
    "age": 25,           # Key: "age"   Value: 25
    "dept": "営業部"     # Key: "dept"  Value: "営業部"
}
```

```
# Keyは文字列が一般的。Valueはどんな型でもOK
scores = {"math": 80, "english": 95, "science": 72}
```

書き方のルール

- { } で全体を囲む
- Key: Value をカンマで区切る
- Keyは通常 "" で囲む文字列
- Valueはどんな型でもOK

空の辞書も作れる

```
empty = {}
```

後から要素を追加できる
→ slide6で詳しく説明

Keyで値を取り出す～何が便利なのか～

辞書[Key] でValueを取り出せます。意味のある名前でアクセスできるのが強みです

```
person = {"name": "山田", "age": 25, "dept": "営業部"}
```

```
# Keyを [ ] で指定してValueを取り出す
print(person["name"])    # → 山田
print(person["age"])     # → 25
print(person["dept"])    # → 営業部
```

存在しないKeyはエラー

```
person["email"]
→ KeyError !
```

辞書にないKeyを指定すると
エラーになる

→ inで確認してから
アクセスするのが安全

f文字列と組み合わせると

```
print(f'名前：{person["name"]}')
→ 名前：山田
```

```
print(f'年齢：{person["age"]}歳')
→ 年齢：25歳
```

読みやすいコードが書ける！

辞書の追加・更新

辞書[Key] = Value の形で追加・更新ができます

```
person = {"name": "山田", "age": 25}

# 要素の追加（存在しないKeyに代入）
person["dept"] = "営業部"
print(person)
# → {"name": "山田", "age": 25, "dept": "営業部"}

# 要素の更新（存在するKeyに再代入）
person["age"] = 26
print(person)
# → {"name": "山田", "age": 26, "dept": "営業部"}
```



追加も更新も書き方は同じ！ Keyが存在しなければ追加、存在すれば上書きされる

keys() / values() / items()

辞書の中身をまとめて取り出すときに使います。特に items()はfor文と組み合わせると便利です

```
person = {"name": "山田", "age": 25, "dept": "営業部"}

# keys() : Keyだけ取り出す
print(person.keys())    # → dict_keys(['name', 'age', 'dept'])

# values() : Valueだけ取り出す
print(person.values())  # → dict_values(['山田', 25, '営業部'])

# items() : KeyとValueをセットで取り出す (for文と相性抜群！)
for key, value in person.items():
    print(f"{key} : {value}")
# → name : 山田  age : 25  dept : 営業部
```



items()はDay2で学んだfor文と組み合わせると全要素を順番に処理できる！

やってみよう:辞書型

自分のプロフィールを辞書で作って操作してみましょう

① 自分のプロフィールを辞書で作ってみよう

```
profile = {  
    "name": "〇〇",  
    "age": 〇〇,  
    "hobby": "〇〇"  
}
```

② Keyを指定して情報を取り出してみよう

```
print(f'名前: {profile["name"]}')  
print(f'趣味: {profile["hobby"]}')
```

③ 新しい項目を追加してみよう

```
profile["dept"] = "〇〇部"  
print(profile)
```

④ items()とfor文で全項目を表示してみよう

```
for key, value in profile.items():  
    print(f'{key}: {value}')
```

まとめ

辞書とは

Key（名前）とValue（値）のペアでデータを管理。{ } で作り、辞書[Key]でValueを取り出す

リストとの使い分け

番号より名前でアクセスしたいとき→ 辞書。
意味のあるKeyをつけることでコードが読みやすくなる

追加・更新

辞書[Key] = Value で追加・更新。Keyが存在しなければ追加、存在すれば上書き

items()とfor文

for key, value in 辞書.items(): で全要素を順番に処理できる

ブール演算

ブール演算とは？

(Boolean :ブーリアン ※ブールとも略されます)

条件に対してTrue(正しい)かFalse(正しくない)かを判定するものです

ブール演算とは？

条件に対してTrue (正しい) か
False (正しくない) かを判定するもの

日常例

「今日は雨が降っている？」
→ True (降っている)
→ False (降っていない)

「年齢は20歳以上？」
→ True / False

```
print(10 > 5)           # → True  
print(10 > 20 and 5 < 10) # → False (andは両方Trueのとき)
```



ブール演算の結果 (True/False) はif文の条件として使います。
Day2で学んだif文と繋がっています！

比較演算子 復習

記号	意味	例
==	等しい	x == 10
!=	等しくない	x != 0
> / <	より大きい / より小さい	score > 80
>= / <=	以上 / 以下	age >= 20

※ 「等しい」は = ではなく == なので注意

条件を組み合わせる(and, or, not)復習

and (かつ)

両方の条件が正しいとき

```
if age >= 20 and status ==  
"社員":
```

or (または)

どちらか一方が正しいとき

```
if score == 100 or is_leader  
== "True":
```

not (~でない)

条件を「反転」させるとき

```
if not is_raining:
```



じつはブール演算子という名前なんです

集合

集合とは？～重複を自動的に除く～

{ }で作りますが辞書と似ているので注意が必要です

Pythonでいう集合とは？

重複が自動的に除かれるデータ構造

リストは重複が残る

```
a = [1, 2, 2, 3, 3]
```

→ [1, 2, 2, 3, 3]

集合は重複が除かれる

```
b = {1, 2, 2, 3, 3}
```

→ {1, 2, 3}

⚠ 辞書との違いに注意！

辞書 { Key: Value }

```
person = {"name": "山田"}
```

集合 { 値 }

```
nums = {1, 2, 3}
```

空の集合は set() で作る！

empty = {} → 辞書

empty = set() → 集合

集合の作り方・追加・削除・確認

{ }・set()で作り、add/remove/inで操作・確認できます

作り方①：{ }で直接作る

A = {1, 2, 3, 4}

作り方②：set()でリストから変換

B = set([1,2,2,3,3]) # → {1,2,3}

作り方③：空の集合

empty = set() # {}は辞書になる！

要素の操作と確認

追加 (add)

A.add(5)

→ {1, 2, 3, 4, 5}

削除 (remove)

A.remove(1)

→ {2, 3, 4, 5}

確認 (in)

print(3 in A) # → True

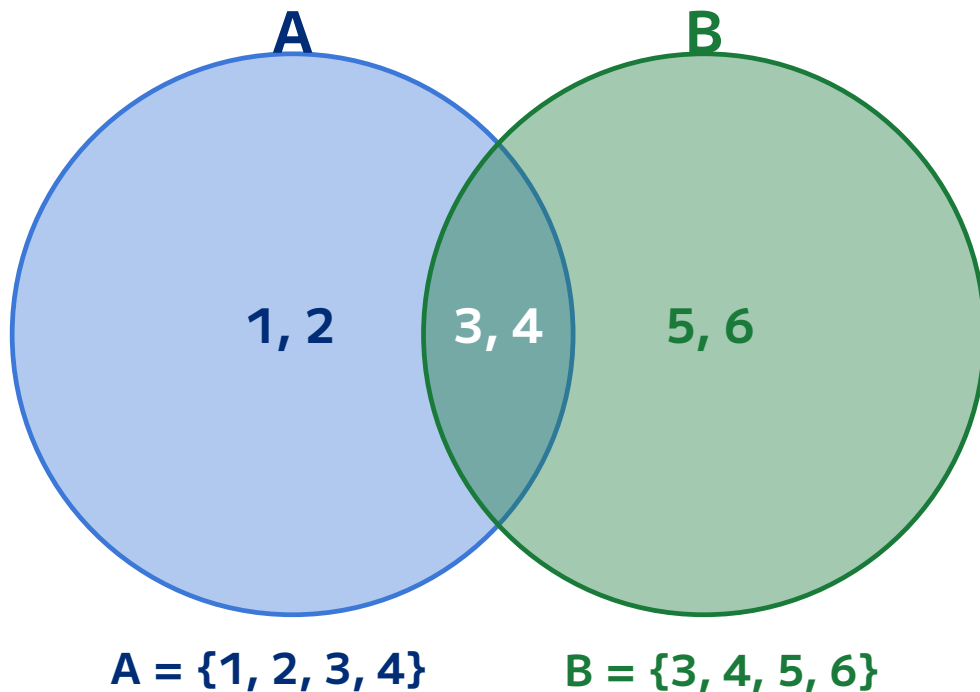
print(9 in A) # → False



inの結果はTrue/False → ブール演算と繋がっています！
if 3 in A: のようにif文でも使えます

集合演算

集合演算には積集合、和集合、差集合の3つがあります



$A = \{1, 2, 3, 4\}$

$B = \{3, 4, 5, 6\}$

& 積集合 : 両方に含まれる

$\text{Print}(A \& B) \rightarrow \{3, 4\}$

| 和集合 : どちらかに含まれる

$\text{Print}(A | B) \rightarrow \{1, 2, 3, 4, 5, 6\}$

- 差集合 : 片方にしかない

$\text{Print}(A - B) \rightarrow \{1, 2\}$

ブール演算との繋がり

and/orと積集合/和集合は同じベン図のイメージで理解できます

ブール演算（条件の判定）

and → 両方の条件を満たす

or → どちらかを満たす

例：age >= 20 and score >= 60

→ 両方TrueのときTrue

→ if文で使う

集合演算（データの抽出）

& → 両方に含まれるデータ

| → どちらかに含まれるデータ

例：A & B

→ 両方のグループに
属するデータ

 andと&、orと|は同じベン図のイメージ！
条件を扱うのがブール演算、データを扱うのが集合演算です

やってみよう:集合

以下のコードを書いて実行してみましょう

```
A = {1, 2, 3, 4, 5}
```

```
B = {4, 5, 6, 7, 8}
```

```
# ① AとBの両方に含まれる数は？
```

```
print(A & B) # → ?
```

```
# ② AまたはBに含まれる全ての数は？
```

```
print(A | B) # → ?
```

```
# ③ AにあってBにない数は？
```

```
print(A - B) # → ?
```

```
# ④ リストから重複を除いてみよう
```

```
data = ["東京","大阪","東京","名古屋","大阪"]
```

```
print(set(data)) # → ?
```

関数

関数とは？～処理に名前をつける～

同じ処理を何度も書かなくて済むようにまとめたものが関数です

関数とは？

同じ処理をまとめて
名前をつけたもの

何度も同じコードを
書かなくて済む！

```
def 関数名():  
    処理
```

実は今まで使ってきた！

今まで使ってきたものは
全部Pythonが用意した関数

```
print() → 表示する関数  
len()   → 長さを返す関数  
str()   → 変換する関数  
int()   → 変換する関数
```

自分でも作れる！

 関数は「処理に名前をつけたもの」。
Pythonが用意した関数だけでなく、自分でも作ることができます！

関数の定義 (def)・基本の形

def キーワードで関数を定義し、関数名() で呼び出します

```
# 関数の定義：def 関数名(): で作る
def greet():
    print("こんにちは！")
```

```
# 関数の呼び出し：関数名() で実行する
greet()    # → こんにちは！
greet()    # → こんにちは！（何度でも使える）
greet()    # → こんにちは！
```

書き方のルール

def のあとに関数名を書く
関数名のあとに () をつける
コロンの (:) を忘れずに
処理はインデント（4マス下げ）

() の意味

() がない → 何も起きない
() がある → 関数が実行される

print と print() の
違いと同じ考え方！

変数との違い

変数はデータを、関数は処理をまとめるものです

変数

値（データ）を入れる箱

```
name = "山田"  
score = 100
```

- データを保存しておく
- 呼び出すと値が返る

イメージ：箱・引き出し

関数

処理（命令）をまとめたもの

```
def greet():  
    print("こんにちは！")
```

- 処理を保存しておく
- 呼び出すと処理が実行される

イメージ：レシピ・機械



変数 = データを入れる箱 関数 = 処理をまとめたもの。
どちらも「名前をつけて後で使う」点は同じ！

引数～関数に値を渡す～

引数を使うと、渡した値に応じて処理を変えることができます

```
# 引数なし：誰に対しても同じ挨拶しかできない
def greet():
```

```
    print("こんにちは！")
```

```
# 引数あり：渡した値に応じて処理が変わる
```

```
def greet(name):    # name が引数
```

```
    print(f"{name}さん、こんにちは！")
```

```
greet("山田")    # → 山田さん、こんにちは！
```

```
greet("佐藤")    # → 佐藤さん、こんにちは！
```

引数とは

関数を呼び出すときに
渡す値のこと

関数の () の中に書く
複数渡すこともできる
def add(a, b):

引数を使うメリット

同じ関数でも
渡す値によって
結果が変わる

→ 1つの関数で
様々な場面に対応できる

戻り値 (return) ～結果を返す～

returnを使うと関数の処理結果を呼び出し元に返すことができます

```
# returnなし：画面に表示するだけ
def greet(name):
    print(f"{name}さん、こんにちは！")
```

```
# returnあり：結果を次の処理に渡せる
def add(a, b):
    return a + b    # 結果を返す
```

```
result = add(10, 20)    # → 30 が返ってくる
print(result)           # → 30
print(result * 2)       # → 60 (さらに計算できる！)
```

returnなし

画面に表示するだけ
結果を他の処理に使えない

returnあり

結果を変数に入れられる
さらに計算・加工できる

関数を使うメリット～再利用性～

関数を使うと修正が1か所で済み、コードが読みやすくなります

❌ 関数なし

```
# 税込価格を3回計算  
print(1000 * 1.1)  
print(2000 * 1.1)  
print(3000 * 1.1)
```

消費税が10%→8%に変わったら
3か所全部直す必要がある😱

100か所あったら...？

✅ 関数あり

```
def tax_price(price):  
    return price * 1.1
```

```
print(tax_price(1000))  
print(tax_price(2000))  
print(tax_price(3000))
```

1.1 を 1.08 に変えるだけ！
修正は1か所だけ😊

💡 関数のメリット：①同じ処理を何度も書かなくていい
②修正が1か所で済む ③処理に名前がつくので読みやすい

やってみよう:関数

以下のコードを書いて実行してみましょう

① 自分の名前を挨拶する関数を作ってみよう

```
def greet(name):  
    print(f"{name}さん、こんにちは！")
```

```
greet("〇〇")    # 自分の名前を入れてみよう
```

② 2つの数を足し算する関数を作ってみよう

```
def add(a, b):  
    return a + b
```

```
result = add(10, 20)  
print(result)    # → 30
```

③ 税込価格を計算する関数を作ってみよう

```
def tax_price(price):  
    return price * 1.1
```

```
print(tax_price(1000))    # → 1100.0
```

まとめ

関数とは

同じ処理をまとめて名前をつけたもの。def で定義し、関数名() で呼び出す

引数

関数を呼び出すときに渡す値。渡した値に応じて処理を変えられる

戻り値 (return)

関数の処理結果を返す。returnがあると結果を変数に入れたりさらに計算できる

メリット

①同じ処理を何度も書かなくていい ②修正が1か所で済む ③コードが読みやすくなる

ファイルの読み込みを学ぼう

超サポ
愉快カンパニー

アシスト

ファイルの読み込みはどんな時に使う？

ファイルの読み込みはどんな時に使う？

Pythonでは「ファイルを読み込む」操作が実務でとても多く登場します。具体的な場面を見てみましょう。

データ集計

Excelやシステムから出力した CSV ファイルを読んで、
売上合計や平均を計算する

ログ解析

システムが自動で書き出したログファイル(.txt)を読んで、エラー
が何件あるかを調べる

設定ファイルの読み込み

プログラムの動作を変えたい時、設定値をファイルに書いておい
て読み込む

一括処理

名前リストや住所一覧のファイルを読んで、メール送信や
ラベル印刷を自動化する

ファイルとは

よく使うファイルの種類

今日学ぶのは「テキストファイル」と「CSV ファイル」の2種類です。

テキストファイル(.txt)

- ・文字だけで構成されたファイル
- ・メモ帳などで開いて中身が見える
- ・ログや設定ファイルに多い

例: sales.txt の中身

1月,150000
2月,230000
3月,180000

CSV ファイル(.csv)

- ・カンマ(,)で値を区切ったテキスト
- ・Excel でも開ける
- ・データのやりとりに広く使われる

名前,年齢,部署
田中,22,営業部
佐藤,25,技術部

ファイルを開く・読む

ファイルを開く: close()を使う書き方

Pythonでファイルを開いたあとは、必ず「閉じる」処理が必要です

基本の書き方(close() を使う場合)

```
f = open("ファイル名", "r", encoding="utf-8")
content = f.read()
f.close()    #自分でファイルを閉じる
```

なぜclose() が必要？

ファイルを開くと、OSがそのファイルを「使用中」と認識する。
close()しないと、ファイルが占有されたままになる。

close() を忘れると

- ・データが正しく保存されないことがある
- ・他のプログラムからファイルが開けなくなる
- ・メモリなどのリソースが無駄に消費される

問題点

処理の途中でエラーが発生すると、
close() がスキップされてしまう

close() の問題点を解消したのが、次の「with 文」です

ファイルを開く: 基本パターン

Pythonでファイルを開くときは、以下の「with 文」を使うのが基本です。

基本の書き方

```
with open("ファイル名", "r", encoding="utf-8") as f:  
    # ここにファイルを読む処理を書く  
    content = f.read()
```

open()

ファイルを開く関数。
使い終わると自動で
閉じてくれる(with の効果)

"r"(モード)

r = read(読む)
書き込むときはw
追記はa

encoding="utf-8"

日本語が文字化けしないための設定。
「utf-8」はほぼ必ず指定すると覚えよう。

with 文のメリット

with ブロックを抜けると自動でファイルが閉じられる。close() の書き忘れを防げる！ → 基本的に with を使って書こう。

テキストファイルを読み込む(readlines)

ファイルの中身を読む方法は複数ありますが、まずはreadlines() を覚えましょう。

★ readlines(): 全行をリストとして読む(最初に覚えるのはこれ！)

greet.txt の中身

こんにちは！
Pythonの練習中です。
3日目もがんばろう！

Pythonコード

```
with open("greet.txt", "r", encoding="utf-8") as f:  
    lines = f.readlines()  
    for line in lines:  
        print(line.strip())
```

実行結果

こんにちは！
Pythonの練習中です。
3日目もがんばろう！

strip() とは？

各行の末尾についている改行(\n)を取り除く。余分な空行を防ぐ。

参考: 他の読み込み方法

read(): 全体を1つの文字列で受け取る readline(): 1行ずつ読む(繰り返し呼ぶたびに次の行へ)

CSVファイルを読む

CSVファイルの読み込み(csv モジュール)

csv モジュールとは

Python に標準で入っているライブラリ。import csv と書くだけで使える。カンマ区切りのデータを行ごとにリストとして読み込める。

members.csv の中身

名前,年齢,部署
田中,22,営業部
佐藤,25,技術部
鈴木,30,総務部

Pythonコード

```
import csv
with open("members.csv", "r",
          encoding="utf-8") as f:
    reader = csv.reader(f)
    for row in reader:
        print(row)
```

実行結果(1行 = 1つのリストになる)

['名前', '年齢', '部署'] ← 1行目(ヘッダー)
['田中', '22', '営業部'] ← row[0] が名前, row[1] が年齢, row[2] が部署
['佐藤', '25', '技術部']
['鈴木', '30', '総務部']

リストのインデックスは 0 から

row[0] が1列目、row[1] が2列目、row[2] が3列目。先頭は 1 ではなく 0 から始まることに注意！

演習問題

演習1:コードを写してみよう



以下のファイルと Pythonコードをそのまま写して実行してみましょう。

① greet.txt を作成する

こんにちは！
Pythonでファイルを読み込んでいます。
3日目の学習、お疲れさまです！

② Pythonコードを書いて実行する

```
with open("greet.txt", "r", encoding="utf-8") as f:  
    lines = f.readlines()  
    for line in lines:  
        print(line.strip())
```

③ 実行するとこう表示されます

こんにちは！
Pythonでファイルを読み込んでいます。
3日目の学習、お疲れさまです！



確認ポイント

・コードを1行ずつ読んで、何をしているか確認しながら入力しよう・strip() を外すとどうなるか試してみよう

演習2: 自分で書いてみよう(テキストファイル)

 以下の内容で fruit.txt を作り、コードの ① ② を埋めて完成させましょう。

fruit.txt の中身

りんご
みかん
ぶどう
バナナ

出力イメージ(目標)

りんご
みかん
ぶどう
バナナ

コードを完成させよう

```
with open("fruit.txt", ① , encoding="utf-8") as f:  
    lines = f. ② ()  
    for line in lines:  
        print(line.strip())
```

 ヒント: ① は読み込みモード(文字列文字)、② は全行をリストで読むメソッド名

演習2: 回答

✓ 正解は以下のとおりです

```
with open("fruit.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()
    for line in lines:
        print(line.strip())
```

解説

"r"	読み込みモード(read の頭文字)。ファイルを読むだけのときはこれ。
readlines()	全行をリスト形式で返す。各行の末尾に改行文字nが含まれている。
strip()	文字列の前後の空白・改行(\n)を取り除く。余分な空行を防ぐ。

演習3: CSVファイルを読み込もう



scores.csv を読み込んで、各行の「名前」と「点数」を表示しましょう。① ② を埋めてください。

scores.csv の中身

名前,点数
田中,85
佐藤,92
鈴木,78

出カイメージ(目標)

田中さんの点数は85 点です
佐藤さんの点数は92 点です
鈴木さんの点数は78 点です

コードを完成させよう

```
import csv

with open("scores.csv", "r", encoding="utf-8") as f:
    reader = csv. ① (f)
    next(reader) # ← 1行目 ("名前,点数" のヘッダー)を読み飛ばす
    for row in reader:
        print(row[ ② ], "さんの点数は", row[1], "点です")
```



ヒント: ① csv. _____ (f) でCSVを読むオブジェクトを作る ② 名前は何列目 (0から数えて) ?

演習3: 回答

✓ 正解は以下のとおりです

```
import csv

with open("scores.csv", "r", encoding="utf-8") as f:
    reader = csv.reader(f)
    next(reader) # ヘッダ行("名前,点数")を1行読み飛ばす
    for row in reader:
        print(row[0], "さんの点数は", row[1], "点です")
```

解説

csv.reader(f)	ファイルオブジェクトfを渡すと、1行ずつリストで読めるオブジェクトを返す。
next(reader)	1行分だけ読み進める関数。最初に呼ぶと「名前点数」のヘッダ一行を読み飛ばせる。
row[0]	リストのインデックスは0から。row[0]=名前、row[1]=点数。何列目かを確認しよう。

まとめ

まとめ

ファイルの種類	テキスト(.txt)とCSV(.csv)。どちらも中身は文字データ。
open()	"r" モードで読み込み。encoding="utf-8" で日本語対応。
with 文	ファイルを自動で閉じてくれる。これを使うのが基本パターン。
readlines()	全行をリストとして読む。strip() で改行(\n)を除去。
csv.reader()	CSV を1行ずつリストで読む。import csv を忘れずに。
next(reader)	1行読み進める。ヘッダ一行を読み飛ばすときに使う。
row[インデックス]	リストは0番から始まる。row[0] が1列目、row[1] が2列目。

変数の種類を学ぼう

変数の種類を知ることによってどんな問題が防げる？

変数の種類を知ることによってどんな問題が防げる？

「変数が使える」だけでなく「変数の種類と仕組み」を知ると、こんな問題が防げます。



なぜかエラーになる

NameError: name "total" is not defined
→ スコープ(有効範囲)を知らないと、
関数の中で作った変数が外から
見えず混乱する



パスワードをうっかり 公開してしまった

password = "abc123" # ← コードに直書き
→ GitHubに上げると全世界に漏れる
環境変数を知ることによって防げる



値を直した場所が 1か所じゃなかった

コード中に "60" が散らばっている
if score >= 60: ...
if point >= 60: ...
→ 定数にまとめれば1か所を修正するだけ

スコープ(有効範囲)

ローカル変数 と グローバル変数

変数は「どこで定義したか」によって、使える場所(スコープ)が決まります。

グローバルスコープ(プログラム全体)

```
company = "Ashisuto" # グローバル変数
```

✓ グローバル変数

- ・関数の外で定義
- ・プログラム全体から参照できる

ローカルスコープ(関数の中)

```
def greet():  
    name = "田中" # ローカル変数  
    # company は外で定義→ 中からでも読める  
    print(company + "の" + name)
```

✓ ローカル変数

- ・関数の中で定義
- ・その関数の中だけで使える
- ✗ 関数の外からは参照できない

スコープのルール:コードで確認しよう

ルール① :関数の中で作った変数は、外からは見えない

```
def calc():  
    result = 100 + 200 # ローカル変数  
  
calc()  
print(result) # ← エラー！関数の外からは見えない
```

実行結果(エラー)

```
NameError:  
  name "result" is not defined
```

ルール② :関数の外で作った変数(グローバル)は、関数の中からも読める

```
company = "Ashisuto" # グローバル変数  
  
def show():  
    print(company) # ← 読める！  
  
show()
```

実行結果

```
Ashisuto
```



ベストプラクティス

関数には「引数」でデータを渡し、結果は「return」で返すのが基本。
グローバル変数の多用は、どこで値が変わるか追いにくなる原因になる。

globalキーワード / よくある落とし穴

関数の中からグローバル変数を「書き換えたい」ときはglobal キーワードが必要です。

✗ よくある間違い

```
count = 0

def add_count():
    count = count + 1 # エラー！
    # Pythonは count をローカルと判断する
    # が、定義前に右辺で使おうとしてエラー

add_count()
```

UnboundLocalError:
local variable 'count' referenced before assignment

⚠ global は便利だが使いすぎ注意

- ・どこで値が変わるかわかりにくくなる → バグの原因になりやすい
- ・基本は 引数で渡す → 関数内で処理 → return で返す の流れを守ろう

✓ global を使う場合

```
count = 0

def add_count():
    global count    # グローバルを使うと宣言
    count = count + 1 # 書き換えOK
```

global 宣言をすることで、関数の中からグローバル変数を変更できる。

定数 (Constants)

定数 (Constants) とは

定数とは

「プログラム中で変更しない値」に名前をつけたもの。Pythonには定数を強制する機能はないが、変数名を 全て大文字 (UPPER_CASE) にするのが慣習。

定数の書き方

定数はすべて大文字で書く

PI = 3.14159

MAX_RETRY = 3

DEFAULT_LANG = "ja"

BASE_URL = "https://api.example.com"

TAX_RATE = 0.10

なぜ定数を使う？

❌ 定数を使わない場合

if score >= 60: ...

if point >= 60: ...

✅ 定数を使う場合

PASS_SCORE = 60

if score >= PASS_SCORE: ...

if point >= PASS_SCORE: ...



定数のメリット

①値を1か所で管理できる(修正は 1箇所 OK) ②コードを読んだとき意味がわかりやすい ③「この値は変えないよ」という意図をチームに伝えられる

環境変数 (Environment Variable)

環境変数 (Environment Variables)

📌 今日のゴール:「環境変数という仕組みがある」ことを知る。使い方は実務で詳しく学ぼう。

🔍 環境変数とは

OSが管理している「名前付きの値」のこと。
Pythonプログラムの外に置いて、
プログラムから読み込んで使う。

💡 どんな時に使う？

パスワードやAPIキーなど、コードに
直接書きたくない値を安全に管理する
ために使われる。

⚠️ なぜ直書きしてはいけない？

コードをチームや外部と共有するとき
パスワードが一緒に見えてしまう。
環境変数にすれば分離できる。

❌ コードに直書き

```
password = "mysecret123" # コードに丸見え
```



✅ 環境変数として外に出す

```
password = os.environ.get("PASSWORD") # 値は外で管理
```

演習問題

演習1:コードを写して動きを確認しよう



以下のコードを写して実行し、それぞれ何が表示されるか確認しましょう。

コード A: スコープの確認

```
company = "Ashisuto"  # グローバル変数
PASS_SCORE = 60      # 定数(慣習でUPPER_CASE)

def show_info():
    dept = "技術部"    # ローカル変数
    print(company)     # グローバルは読める
    print(dept)
    print(PASS_SCORE)

show_info()
```

実行すると...

Ashisuto / 技術部 / 60

コード B: 外からローカル変数を見ると?

```
def show_info():
    dept = "技術部"

show_info()
print(dept)  # ← これを追加して実行
```

実行すると...

NameError: name "dept" is not defined

👉 エラーメッセージと照らし合わせて
「ローカル変数は外から見えない」を体感しよう

演習2: 穴を埋めてプログラムを完成させよう



社員の給与計算プログラムです。① ② を埋めて完成させましょう。

① = 1000 # 「基本給」を表す定数(変わらない値)→大文字で書く

```
def calc_salary(hours):
```

```
    ② salary = BASE_PAY * hours # 関数の外のBASE_PAYを書き換えたい
```

```
    salary = BASE_PAY * hours
```

```
    return salary
```

```
pay = calc_salary(160)
```

```
print("今月の給与:", pay, "円") # → 今月の給与: 160000 円
```



ヒント

① 定数名は「基本給」=BASE_PAY、値は 1000。大文字で書くのが慣習。

② 関数の中からグローバル変数を書き換えるためのキーワードは何でしたか？

演習2: 回答

✓ 正解は以下のとおりです

```
BASE_PAY = 1000 # ① 定数 (UPPER_CASE で書く)

def calc_salary(hours):
    global BASE_PAY # ② global キーワードで宣言
    salary = BASE_PAY * hours
    return salary

pay = calc_salary(160)
print("今月の給与:", pay, "円") # → 今月の給与: 160000 円
```

解説

① `BASE_PAY = 1000`

定数は UPPER_CASE (すべて大文字) で書くのが Python の慣習。変更しない値はこの形に。

② `global BASE_PAY`

関数内でグローバル変数を書き換えるには `global` 宣言が必要。読むだけなら不要だが、代入する場合は必ず宣言する。

なぜ `global` が必要？

Python は関数内で代入が行われると「ローカル変数だ」と判断するため、グローバルと明示しないと `UnboundLocalError` が起きる。

まとめ

演習2: 回答

ローカル変数	関数の中で定義。その関数の中だけで使える。外からは見えない。
グローバル変数	関数の外で定義。プログラム全体から参照できる。多用は避ける。
global キーワード	関数の中でグローバル変数を書き換えたいときに宣言する。
定数 (Constants)	変えない値に名前をつける。変数名をUPPER_CASE で書くのが慣習。
環境変数	OSが管理する変数。パスワード・APIキーなどコードに直書きしたくない値を保管。
環境変数のイメージ	コードに直書きしたくない値をOS側で管理する仕組み。パスワード等の保管に使う。詳細は実務で。